Integración de Sistemas

Semana 10. Observabilidad
y trazas distribuidas

Sistemas de Información — PUCE
Docente: Damián Nicolalde

Semana 10. Observabilidad y trazas distribuidas

Objetivo: Comprender los tres pilares clásicos de la observabilidad (logs, métricas y trazas), dominar el estándar abierto OpenTelemetry para instrumentar aplicaciones distribuidas, visualizar e interpretar trazas reales en Jaeger, definir KPIs de integración con rigor (SLI, SLO, SLA, error budget), y usar la correlación de trazas como herramienta central de debugging distribuido.

Resultado de aprendizaje asociado: Implementar un ecosistema de microservicios expuestos mediante APIs REST y gestionados con API Gateway y un bus de eventos, incorporando contratos, pruebas, observabilidad y seguridad para asegurar escalabilidad y desempeño.

Nota de continuidad: esta semana no tiene contenido previo en contenido/ — se construyó desde el sílabo oficial. Conecta directamente con la Semana 6, que ya introdujo conceptualmente el correlation-id y los logs estructurados sobre la Saga de Pedidos. Esta semana lleva ese concepto a la práctica real, instrumentando con el SDK de OpenTelemetry y observando trazas reales en Jaeger.

Introducción

A lo largo de nueve semanas has construido un ecosistema cada vez más distribuido: una API REST de Productos (Semanas 1-2), un servicio gRPC (Semana 3), un servidor GraphQL (Semana 4), una integración SOA con mensajería (Semana 5), una Saga de cuatro microservicios comunicados por eventos (Semana 6), contratos formales con JSON Schema (Semana 7), un API Gateway (Semana 8) y una disciplina completa de pruebas y contract testing (Semana 9). Cada pieza, por separado, se entiende leyendo su código. Pero en el momento en que una petición real cruza varias de estas piezas —como ocurre en la Saga de Pedidos de la Semana 6— surge una pregunta que ningún log aislado puede responder: ¿qué pasó exactamente con esta solicitud en particular, en todo el sistema, de principio a fin?

Esta pregunta es el problema central que resuelve la observabilidad. No es un lujo de sistemas "muy grandes"; aparece en el momento exacto en que una solicitud deja de ser atendida por un único proceso y empieza a atravesar una red de servicios independientes, cada uno con su propio ciclo de vida y sus propios fallos parciales. Un sistema monolítico se depura con un debugger paso a paso y una pila de llamadas local. Un sistema distribuido no tiene una pila de llamadas única — la tiene fragmentada en la memoria de cada proceso, en distintos momentos, a veces en distintas máquinas. La observabilidad es la disciplina de reconstruir esa pila fragmentada después de que ocurrió, y de poder preguntar por qué ocurrió lo que ocurrió sin haber anticipado exactamente esa pregunta al escribir el código.

Esta semana desarrolla esa disciplina en cinco pasos: los tres pilares clásicos y por qué ninguno basta solo (10.1); OpenTelemetry, el estándar abierto que unifica cómo se generan e intercambian estos datos (10.2); el stack de backends que los reciben y visualizan — Jaeger, ELK, Grafana (10.3); cómo traducir la observación en objetivos de negocio medibles — SLI, SLO, SLA,

error budget (10.4); y la habilidad más práctica de la semana: usar una traza distribuida real para encontrar la causa de un problema invisible con solo logs sueltos (10.5).

La experiencia de laboratorio retoma directamente el proyecto de la Semana 6: instrumentarás primero nestjs-productos-api con el SDK de OpenTelemetry para ver tu primer trace en Jaeger, y luego instrumentarás los cuatro microservicios de la Saga de Pedidos para obtener un único trace distribuido que atraviesa los cuatro servicios a través de RabbitMQ.

Vale la pena detenerse un momento en por qué esta semana se ubica precisamente aquí, después de la Semana 9 (pruebas y contract testing) y antes de la Semana 11 (seguridad). Las pruebas automatizadas de la Semana 9 te dan confianza sobre el comportamiento del sistema ANTES de desplegarlo — verifican que el código hace lo que se supone que debe hacer, bajo condiciones controladas y conocidas de antemano. La observabilidad de esta semana resuelve el problema complementario: qué hacer cuando el sistema ya está desplegado y se comporta de una forma que ninguna prueba anticipó, porque las condiciones reales de producción (volumen de tráfico real, combinaciones de datos reales, fallos de infraestructura reales) son, casi por definición, imposibles de replicar exhaustivamente en un entorno de pruebas. Un sistema con una suite de pruebas excelente pero sin observabilidad sigue siendo, en producción, una caja negra en el momento exacto en que más se necesita verla por dentro. Y la seguridad de la Semana 11, a su vez, depende en buena medida de la observabilidad de esta semana: detectar un acceso no autorizado o un patrón de tráfico anómalo exige exactamente el mismo tipo de instrumentación —logs estructurados, trazas, métricas— que esta semana desarrolla desde una motivación de confiabilidad operativa, no de seguridad. Las tres semanas, en conjunto, describen el ciclo de vida completo de la confianza en un sistema distribuido: verificarlo antes de desplegar (Semana 9), observarlo mientras corre (Semana 10), y protegerlo de actores malintencionados (Semana 11).

10.1. Logs, métricas y trazas

Los tres pilares clásicos

La literatura de observabilidad organiza las señales de un sistema en vivo en tres tipos fundamentales de datos, conocidos como los "tres pilares":

Pilar	Qué es	Pregunta que responde
Logs	Registros discretos de eventos, con timestamp y contexto	¿Qué pasó exactamente, en detalle, en este momento?
Métricas	Valores numéricos agregados a lo largo del tiempo	¿Cómo se comporta el sistema en su conjunto, y cómo cambia?
Trazas	El recorrido completo de una solicitud a través de todos los servicios que tocó	¿Por dónde pasó exactamente ESTA solicitud, y cuánto tardó cada tramo?

Cada pilar tiene una fortaleza que compensa la debilidad de los otros dos. Los logs dan detalle exhaustivo, pero son difíciles de correlacionar a gran escala. Las métricas son extremadamente baratas de almacenar y agregar, y permiten ver tendencias y anomalías de un vistazo — pero, por estar agregadas, pierden el detalle de qué solicitud individual causó el pico. Las trazas dan la vista más rica del recorrido de una solicitud concreta, pero instrumentar trazas para todas las solicitudes de un sistema de alto volumen puede ser costoso, por lo que en la práctica se aplican técnicas de muestreo (sampling).

Monitoreo vs. observabilidad

Monitoreo es la práctica de recolectar métricas predefinidas y compararlas contra umbrales conocidos de antemano, generando alertas cuando se cruza un umbral. Responde preguntas que ya sabías que ibas a necesitar hacer: ¿está el CPU por encima del 80%? Observabilidad es la propiedad de un sistema que permite a un humano formular y responder preguntas que no anticipó al instrumentar el sistema. El ejemplo canónico: ¿por qué los pedidos con cupón de descuento Y pago con tarjeta Y ubicados en Guayaquil tardan más en la fase de reserva de inventario? Nadie hubiera puesto una métrica exactamente para ese cruce de condiciones. Un sistema observable permite responderla igual, porque los datos crudos contienen suficiente información para reconstruir cualquier corte que se te ocurra hacer después del hecho.

Diagrama 1. Los 3 pilares de la observabilidad y la pregunta que responde cada uno.



Por qué ninguno de los tres basta por sí solo

Un escenario real lo ilustra: una métrica muestra un pico de latencia p99 a las 14:32. La métrica dice que hubo un problema y cuándo, pero no cuál solicitud lo causó ni por qué. Buscas en los logs de ese minuto y encuentras cientos de líneas de cuatro microservicios, sin forma evidente de saber cuáles pertenecen a la misma solicitud lenta. Aquí entra la traza: si cada solicitud lleva un trace_id propagado, filtras Jaeger por ese rango de tiempo, encuentras las trazas más lentas, abres una y ves de un vistazo que el 90% del tiempo se fue en un solo servicio. La métrica dio la alarma; la traza dio el diagnóstico; y para el detalle exacto del error, vuelves al log de ese trace_id — los tres pilares correlacionados por el mismo identificador son lo que constituye observabilidad real.

Aspecto	Logs	Métricas	Trazas
Volumen de almacenamiento	Alto	Muy bajo	Medio-alto (muestreable)
Granularidad	Máxima	Mínima (agregado)	Por solicitud individual
Ideal para	Diagnóstico detallado puntual	Tendencias, alertas, dashboards	Entender el recorrido y latencia
Costo de instrumentar	Bajo	Bajo-medio	Medio (requiere propagación de contexto)

El costo de no invertir en observabilidad: una perspectiva económica

Es tentador tratar la observabilidad como un gasto de infraestructura postergable — "lo agregamos cuando tengamos un problema real". Esta perspectiva ignora una asimetría de costos bien documentada: el costo de instrumentar un sistema DESDE EL DISEÑO es marginal (unas pocas líneas de configuración, un archivo tracing.ts), mientras que el costo de reconstruir observabilidad DESPUÉS de que un sistema ya está en producción, bajo la presión de un incidente activo sin visibilidad, es órdenes de magnitud mayor: hay que identificar manualmente los puntos de instrumentación, desplegar cambios con el sistema ya inestable, y en el peor caso

el incidente se resuelve por fuerza bruta (reiniciar todo, revertir el último despliegue "por si acaso") sin entender la causa raíz, dejando la puerta abierta a que el mismo problema recurra semanas después sin que nadie lo reconozca como el mismo problema. Instrumentar temprano no es una opción entre varias igualmente válidas — es, medido en el costo total esperado a lo largo de la vida del sistema, casi siempre la decisión económicamente correcta.

Ejercicio resuelto

Enunciado: un compañero dice: "no necesitamos trazas, con logs estructurados y buenos índices en Elasticsearch nos alcanza". Evalúa el argumento.

Solución: parcialmente cierto para sistemas con pocos servicios (2-3), donde correlacionar logs con un correlation_id es manejable. No escala: (1) reconstruir el árbol de llamadas a partir de logs sueltos exige reconstruir manualmente lo que una traza ya representa nativamente; (2) es difícil visualizar de un vistazo cuál tramo fue el cuello de botella con 6+ servicios; (3) los logs no capturan automáticamente la propagación de contexto entre procesos, con riesgo de perder el hilo en un salto asíncrono como RabbitMQ. Los logs siguen siendo indispensables para el detalle fino, pero no sustituyen a las trazas para el recorrido completo.

Sampling: por qué no se traza el 100% en producción

Instrumentar trazas tiene un costo real: en un sistema con decenas de miles de solicitudes por segundo, exportar el 100% de las trazas puede saturar la red, el Collector y el backend, además de generar un costo de almacenamiento considerable. Por eso, en producción se aplica muestreo (sampling):

- Head-based sampling: la decisión de exportar o no se toma al inicio de la traza, antes de saber cómo terminará. Simple y barato, pero si una traza termina en error y no fue elegida, se pierde para siempre.
- Tail-based sampling: la decisión se toma después de ver la traza completa, permitiendo reglas inteligentes como conservar siempre las trazas con error o las más lentas. Más costoso operacionalmente, pero muy superior en valor diagnóstico.
- Muestreo por prioridad de negocio: ciertas rutas críticas (ej. POST /pedidos) siempre se trazan al 100% independientemente de la estrategia general.

Enunciado (2do ejercicio): un equipo implementa head-based sampling al 1%. Un cliente reporta un error intermitente en POST /pedidos. Jaeger no muestra ninguna traza con ese error. ¿Es evidencia de que el error no existe?

Solución: no. Con head-based al 1%, la probabilidad de que la traza específica de ese error haya sido elegida es también ~1% — la ausencia de evidencia en Jaeger no es evidencia de ausencia del problema. Esto motiva el tail-based sampling, que conservaría siempre las trazas con error.

Taller en grupo — Clasificar señales con IA

Modalidad: parejas, 20 minutos, Nivel de IA: 2 (borrador). Tomen 5 preguntas reales sobre su Saga de Pedidos de la Semana 6.

Prompt sugerido: "Para cada una de estas 5 preguntas sobre un sistema distribuido, indica si se responde principalmente con logs, con métricas, con trazas, o con una combinación, y justifica en una frase."

Verificación obligatoria: para cada pregunta clasificada como "traza", confirmen que efectivamente necesita ver el recorrido completo de UNA solicitud.

10.2. OpenTelemetry

Qué es y por qué existe

OpenTelemetry (OTel) es un estándar abierto, gobernado por la CNCF, que define de forma vendor-neutral cómo instrumentar código para generar logs, métricas y trazas, y cómo exportarlos hacia cualquier backend sin acoplar el código de la aplicación a un proveedor específico. Antes de OpenTelemetry (nacido en 2019 de la fusión de OpenTracing y OpenCensus), cada proveedor tenía su propio SDK propietario; migrar de proveedor implicaba reinstrumentar toda la aplicación. OpenTelemetry resuelve esto: instrumentas una sola vez con el SDK de OTel, y decides el backend de destino cambiando solo el exporter.

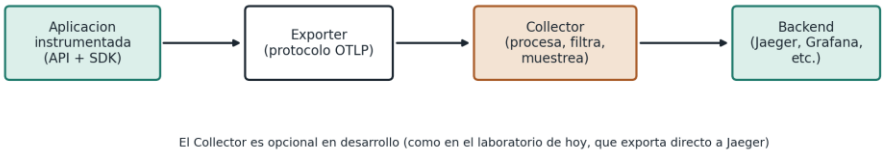
Componentes de OpenTelemetry

Componente	Rol
API	Interfaces que usa el código de la aplicación para crear spans/métricas/logs — estable, no depende de ningún backend.
SDK	Implementación concreta de la API: procesa, agrupa (batching) y prepara los datos.
Instrumentación automática	Librerías que enganchan automáticamente frameworks conocidos (Express, HTTP, DB) sin código manual.
Instrumentación manual	Código explícito para spans/atributos que la auto-instrumentación no puede inferir.
Exporter	Traduce los datos del SDK al formato/protocolo de un backend específico (ej. OTLP/HTTP hacia Jaeger).
Collector	Proceso intermedio opcional que recibe datos de muchas apps, los procesa y reenvía a uno o varios backends.

Anatomía de un trace

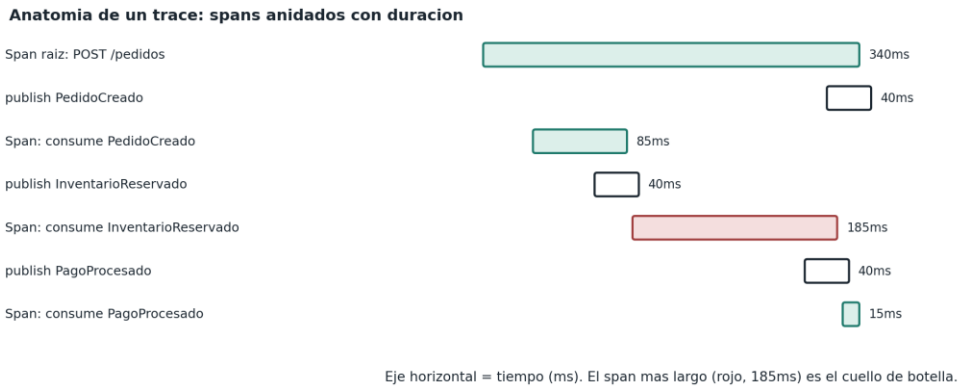
Un trace representa el recorrido completo de una solicitud, identificado por un trace_id único (128 bits) que se propaga a todos los servicios involucrados. Dentro de un trace, cada unidad de trabajo se representa como un span, con: un span_id propio, una referencia opcional a un parent_span_id, un nombre descriptivo, timestamps de inicio/fin, atributos (pares clave-valor), eventos, y un status (OK, error, o sin establecer).

Diagrama 2. Arquitectura de OpenTelemetry: aplicación instrumentada → SDK → Exporter → Collector (opcional) → backend.



El Collector es opcional en desarrollo (como en el laboratorio de hoy, que exporta directo a Jaeger)

Diagrama 3. Anatomía de un trace: spans anidados padre-hijo con su duración exacta, representados como barras tipo Gantt.



Propagación de contexto: el corazón técnico de las trazas distribuidas

Para que un trace atraviese varios servicios y siga siendo el "mismo" trace, el contexto debe propagarse junto con cada solicitud. El estándar W3C Trace Context, recomendación oficial del W3C, define el header HTTP traceparent:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             version    trace_id (32 hex)              span_id (16 hex) flags
```

En comunicación síncrona (HTTP), la propagación es sencilla: el cliente inyecta el header traceparent en la petición saliente y el servidor lo lee al recibirla. La mayoría de auto-instrumentaciones HTTP lo hacen automáticamente. En comunicación asíncrona (colas como RabbitMQ, como en la Semana 6), es más delicada: no hay un request/response directo. La solución es incluir el contexto de traza en los headers del mensaje AMQP; cuando la auto-instrumentación no lo hace automáticamente, es responsabilidad del desarrollador propagarlo manualmente con propagation.inject()/propagation.extract() de la API de OpenTelemetry.

Tipos de span (SpanKind) y convenciones semánticas

OpenTelemetry define un conjunto de SpanKind que clasifican el rol de un span: SERVER (atiende una solicitud entrante), CLIENT (llamada saliente hacia otro servicio), PRODUCER (publica un mensaje asíncrono) y CONSUMER (recibe y procesa un mensaje asíncrono). Los backends de visualización usan el SpanKind para dibujar las relaciones entre spans de distintos servicios. Además, OpenTelemetry define convenciones semánticas: nombres estandarizados de atributos comunes (http.method, http.status_code, db.system, messaging.destination) que las herramientas de análisis saben interpretar automáticamente.

Instrumentación real en NestJS: el archivo tracing.ts

En la práctica, instrumentar un servicio NestJS sigue un patrón consistente: un archivo separado (tracing.ts) que configura e inicia el SDK, importado como la PRIMERA línea de main.ts, antes de cualquier otra importación — crítico porque la auto-instrumentación parchea módulos de

Node.js en el momento en que se cargan; si Nest ya se cargó antes, la instrumentación llega tarde.

```
// tracing.ts - debe importarse ANTES que cualquier otra cosa en main.ts
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentations-
node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';
import { resourceFromAttributes } from '@opentelemetry/resources';

const sdk = new NodeSDK({
  resource: resourceFromAttributes({ 'service.name': 'nestjs-productos-api' }),
  traceExporter: new OTLPTraceExporter({ url: 'http://localhost:4318/v1/traces' }),
  instrumentations: [getNodeAutoInstrumentations()],
});

sdk.start();
```

```
// main.ts
import './tracing'; // PRIMERA linea
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(3000);
}
bootstrap();
```

Ejercicio resuelto

Enunciado: en la Saga de Pedidos, PedidoService publica un evento a RabbitMQ e InventarioService lo consume 200ms después. Si no se propaga el contexto de traza en los headers del mensaje, ¿qué verás en Jaeger?

Solución: dos trazas separadas, no una: una que termina en PedidoService y otra completamente nueva que empieza en InventarioService, con un trace_id distinto generado desde cero. Se pierde por completo la capacidad de ver el flujo end-to-end como una sola unidad.

Segundo ejercicio resuelto: decidir dónde instrumentar manualmente

Enunciado: la auto-instrumentación de OpenTelemetry ya captura automáticamente cada llamada HTTP entrante y saliente, y cada consulta a la base de datos, como spans separados con buena granularidad. ¿En qué situación seguiría siendo necesario agregar instrumentación MANUAL, si la automática ya cubre HTTP y base de datos?

Solución: la auto-instrumentación captura las fronteras técnicas (llamadas de red, consultas SQL) pero no conoce el significado de negocio de tu código. Si dentro de un mismo handler HTTP tienes tres pasos de lógica de negocio secuenciales sin ninguna llamada de red entre ellos (por ejemplo: validar reglas de negocio, calcular un descuento, y decidir si aplica un flag

experimental), la auto-instrumentación los ve todos como parte de un único span HTTP indiferenciado. Si uno de esos tres pasos es el que consume la mayor parte del tiempo, sin instrumentación manual no hay forma de saberlo desde la traza — solo verías "el span HTTP tardó 800ms", sin ninguna pista de cuál de los tres pasos internos fue el responsable. La instrumentación manual llena exactamente ese vacío: expone la estructura de negocio que la auto-instrumentación, por diseño, no puede inferir.

Taller en grupo — Diseñar instrumentación manual con IA

Modalidad: parejas, 25 minutos, Nivel de IA: 2 (borrador). Tomen el controller crearPedido de nestjs-pedido-service.

Prompt sugerido: "Dado este método de un controller NestJS que valida datos, guarda un pedido y publica un evento a RabbitMQ, propone qué spans manuales y atributos agregarías con la API de OpenTelemetry para diferenciar la validación, el guardado y la publicación dentro del span HTTP principal."

Verificación obligatoria: confirmen que cada span propuesto representa una unidad de trabajo con sentido semántico propio, no un span por cada línea de código.

10.3. Jaeger + ELK + Grafana

Jaeger: backend de visualización de trazas

Jaeger es un sistema de trazado distribuido de código abierto, creado en Uber y donado a la CNCF. Su arquitectura típica: Agente/Collector (recibe spans exportados), Query (consulta el almacenamiento) y UI (interfaz web en el puerto 16686 donde se buscan y visualizan trazas como diagrama de Gantt). Para el laboratorio se usa la imagen jaegertracing/all-in-one, que empaqueta los tres componentes en un contenedor — ideal para desarrollo, aunque en producción se despliegan por separado con almacenamiento persistente como Elasticsearch o Cassandra.

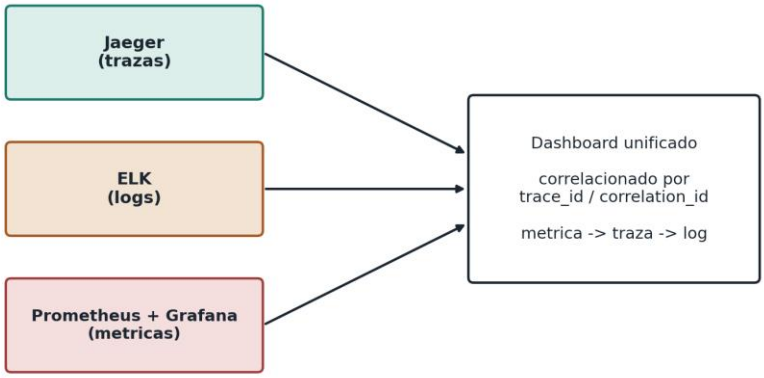
ELK: centralización y búsqueda de logs

ELK es el acrónimo de Elasticsearch (motor de búsqueda donde se almacenan los logs), Logstash/Filebeat (recolecta y transforma logs) y Kibana (interfaz de visualización y búsqueda). Cuando los logs se centralizan con el mismo correlation_id/trace_id como campo indexado, se puede buscar por ese identificador y ver todas las líneas de log de todos los servicios que participaron en una solicitud — el complemento natural de una traza para el detalle textual completo.

Grafana y Prometheus: visualización de métricas

Prometheus recolecta métricas mediante pull (consulta periódicamente un endpoint /metrics de cada servicio) y las almacena como series de tiempo. Grafana consulta Prometheus (y otras fuentes, incluyendo Elasticsearch y Jaeger) para construir dashboards: gráficos de latencia, tasas de error, throughput, y alertas configurables.

Diagrama 4. Stack de observabilidad completo: Jaeger + ELK + Grafana/Prometheus convergiendo en un dashboard correlacionado por trace_id.



Cómo se combinan en la práctica

Un flujo típico de investigación de un incidente: (1) una alerta de Grafana dispara: la tasa de error subió al 8%; (2) el ingeniero abre Grafana e identifica la ventana de tiempo exacta; (3) va a Jaeger, filtra por servicio y ventana, ordena por duración/error; (4) abre una traza específica, ve qué span falló y su trace_id; (5) busca ese trace_id en Kibana y encuentra el mensaje de error completo. Este flujo "métrica → traza → log" es la aplicación práctica de 10.1.

Herramienta	Pilar principal	Rol típico
Jaeger	Trazas	Visualizar el recorrido y latencia de solicitudes individuales
ELK	Logs	Buscar y correlacionar el detalle textual de eventos
Prometheus + Grafana	Métricas	Dashboards de tendencia y alertas basadas en umbrales

Autoalojado vs. SaaS de observabilidad

El stack Jaeger + ELK + Grafana/Prometheus es autoalojado (self-hosted): la organización opera, escala y mantiene cada componente. La alternativa es contratar un proveedor SaaS de observabilidad (Datadog, New Relic, Honeycomb, Grafana Cloud), que ofrece las mismas capacidades como servicio gestionado, cobrando por volumen de datos ingeridos.

Criterio	Autoalojado	SaaS
Costo inicial	Bajo (software libre)	Puede ser alto a volumen
Costo operativo	Alto (hay que operar la infraestructura)	Bajo (el proveedor la opera)
Control de datos	Total, nunca sale de tu infraestructura	Los datos viajan al proveedor
Velocidad de adopción	Más lenta	Muy rápida (SDK + API key)

Gracias a que toda la instrumentación se hizo con OpenTelemetry (estándar vendor-neutral), migrar de uno a otro es, idealmente, solo cambiar la configuración del exporter, sin tocar el código de instrumentación — la razón de negocio más citada para adoptar OpenTelemetry desde el principio.

Ejercicio resuelto

Enunciado: tu docente pregunta: "si Grafana ya me dice que hay un problema, ¿para qué necesito además Jaeger?". Responde con un ejemplo concreto.

Solución: Grafana te dice QUE la latencia p95 subió a 3 segundos a las 14:32, pero no CUÁL de los cuatro microservicios de la Saga causó el aumento. Sin Jaeger, revisarías logs de los cuatro a mano. Con Jaeger, filtras trazas lentas en esa ventana y ves en el Gantt de una sola traza que el span de InventarioService ocupó 2.6 de los 3 segundos totales.

Segundo ejercicio resuelto: elegir el backend adecuado

Enunciado: un startup de 4 personas necesita observabilidad básica para su primer producto en producción, con presupuesto muy limitado y sin nadie dedicado a operar infraestructura. Una empresa mediana con equipo de plataforma dedicado necesita observabilidad para 40 microservicios con requisitos estrictos de residencia de datos (los datos no pueden salir del país). ¿Qué recomendarías a cada una?

Solución: para el startup, un SaaS con capa gratuita (o de bajo costo) es preferible: no tienen capacidad operativa para mantener Jaeger/ELK/Grafana autoalojados, y la velocidad de adopción importa más que el control fino de costos a ese volumen pequeño. Para la empresa mediana, autoalojado (o un SaaS con garantías contractuales explícitas de residencia de datos) es la opción correcta: el requisito de residencia de datos puede ser una restricción legal no negociable, y ya cuentan con el equipo de plataforma necesario para operar la infraestructura. En ambos casos, instrumentar con OpenTelemetry desde el día uno preserva la opción de migrar más adelante sin reinstrumentar.

Taller en grupo — Diseñar un dashboard con IA

Modalidad: equipos de 3-4, 25 minutos, Nivel de IA: 2 (borrador).

Prompt sugerido: "Para el flujo Crear Pedido de nuestra Saga de 4 microservicios, propone qué 4-5 paneles incluirías en un dashboard de Grafana para monitorear su salud en producción, nombrando la métrica exacta de cada panel."

Verificación obligatoria: para cada panel, decidan si la métrica es calculable con datos que YA generan (spans/logs existentes) o requiere instrumentación adicional.

10.4. KPIs de integración: latencia, throughput, SLOs, SLIs

Del dato crudo al objetivo de negocio

La disciplina de Site Reliability Engineering (SRE), popularizada por Google, define un vocabulario preciso para conectar la observación técnica con compromisos de negocio:

- SLI (Service Level Indicator): un indicador medible, cuantitativo, derivado directamente de las señales de observabilidad. Ejemplo: el p95 de la latencia de POST /pedidos.
- SLO (Service Level Objective): un objetivo sobre ese indicador, expresado como umbral y período. Ejemplo: 95% de las peticiones en menos de 300ms, en 30 días.
- SLA (Service Level Agreement): un compromiso contractual, con consecuencias económicas o legales, acordado con un cliente externo — típicamente más laxo que los SLO internos.

La relación es de contención: el SLI es la materia prima medible; el SLO es un objetivo interno sobre ese SLI; el SLA es una promesa externa, generalmente más conservadora que el SLO interno.

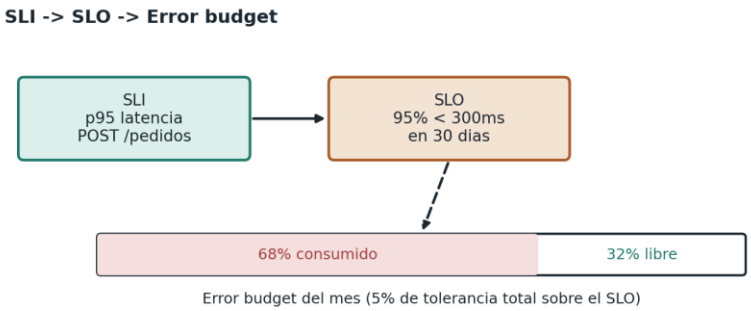
Latencia y throughput

Latencia: el tiempo de procesar una solicitud individual, reportado como percentiles (p50, p95, p99) en vez de promedio (que oculta valores extremos). Un p99 de 2 segundos significa que el 1% de las solicitudes más lentas tarda 2 segundos o más — en un sistema con millones de solicitudes diarias, ese 1% sigue siendo enorme, razón por la que el p99 es el número que importa para la experiencia de usuario en la cola. Throughput: la cantidad de solicitudes/eventos procesados por unidad de tiempo (RPS). Ambas se derivan directamente de las trazas: la duración del span raíz da la latencia; contar spans raíz nuevos por segundo da el throughput.

Error budget

Si el SLO establece 99.5% de disponibilidad en 30 días, el error budget es el 0.5% restante — el margen de fallo permitido antes de incumplir el objetivo. Mientras el error budget no se agote, el equipo tiene libertad para tomar riesgos calculados (desplegar más seguido); en cuanto se agota, la prioridad cambia automáticamente hacia estabilizar el sistema. Es una forma objetiva de resolver la tensión entre velocidad de entrega y estabilidad.

Diagrama 5. Relación SLI → SLO → error budget, con una barra de presupuesto de error consumiéndose.



Concepto	Ejemplo sobre la Saga de Pedidos
SLI	p95 de la duración total del trace Crear Pedido
SLO	95% de los pedidos completan su Saga en menos de 2 segundos, 30 días
Error budget	5% de los pedidos pueden tardar más de 2 segundos antes de incumplir el SLO
SLA (si existiera)	99% de los pedidos confirmados en menos de 5 segundos — más laxo que el SLO interno

Alertas multi-ventana multi-tasa de consumo (burn rate)

Una alerta basada en error budget que solo avisa al llegar al 100% consumido llega demasiado tarde para actuar. La práctica SRE moderna usa alertas basadas en tasa de consumo (burn rate): qué tan rápido, respecto al ritmo presupuestado, se está gastando el error budget. Un burn rate de 1x significa que el presupuesto se agotará justo al final del mes; un burn rate de 10x significa que se agotará en solo 3 días — señal de alarma mucho más urgente aunque el porcentaje consumido hasta ahora parezca bajo. La técnica multi-window multi-burn-rate (documentada por Google) combina varias ventanas de tiempo con distintos umbrales para distinguir un pico transitorio de una degradación sostenida.

Burn rate	Interpretación	Tiempo hasta agotar el budget mensual
1x	Ritmo presupuestado normal	~30 dias
6x	Degradacion notable	~5 dias
14.4x	Degradacion severa	~2 dias
36x	Incidente grave en curso	~20 horas

Apdex: una métrica compuesta de satisfacción

Además de percentiles, algunos equipos usan el Apdex (Application Performance Index): clasifica cada solicitud como satisfecha (tiempo <= T), tolerante (T < tiempo <= 4T) o frustrada (tiempo > 4T), y se calcula como (satisfechas + tolerantes/2) / total. Es menos preciso que examinar percentiles directamente, pero su valor está en la comunicación: un solo número entre 0 y 1 es más fácil de poner en un dashboard ejecutivo que una distribución completa.

Casos reales de SLOs en la industria

Aunque los números exactos de SLOs internos de grandes empresas rara vez se publican con precisión, el patrón general es bien conocido y documentado en la literatura de SRE: servicios de infraestructura crítica (bases de datos gestionadas, sistemas de autenticación, pasarelas de pago) suelen tener SLOs de disponibilidad muy altos, del orden de 99.9% a 99.99%, precisamente porque son dependencias de las que dependen muchos otros sistemas río abajo — un solo minuto de caída se multiplica en impacto a través de toda la cadena de servicios que dependen de ellos. Servicios de cara al usuario menos críticos (una función de recomendaciones, una sección de "productos relacionados") suelen tolerar SLOs más bajos, del orden de 99% a 99.5%, porque su falla temporal degrada la experiencia sin impedir la operación central del negocio (un usuario puede completar una compra aunque no vea recomendaciones).

Tipo de servicio	SLO de disponibilidad típico	Justificación
Autenticación / pagos (crítico)	99.9% - 99.99%	Un fallo bloquea la operación central del negocio para todos los usuarios
API principal de un producto	99.5% - 99.9%	Degrada la experiencia central pero rara vez la bloquea por completo
Recomendaciones / analítica (no crítico)	99% - 99.5%	Su falla temporal es tolerable; el negocio sigue funcionando

La lección práctica de esta comparación es que **no todos los servicios necesitan el mismo SLO** — definir el mismo objetivo estricto para todo el ecosistema desperdicia esfuerzo de ingeniería en servicios donde no aporta valor de negocio proporcional, mientras que definir el mismo objetivo laxo para todo arriesga la estabilidad de las piezas verdaderamente críticas. Priorizar dónde invertir en confiabilidad, basándose en el impacto real de negocio de cada servicio, es en sí mismo un ejercicio de ingeniería tan importante como la instrumentación técnica.

Ejercicio resuelto

Enunciado: el SLO es "p99 de latencia de GET /productos < 500ms, 30 días". A los 10 días ya se consumió el 95% del error budget. ¿Qué decisión sugiere este dato?

Solución: congelar temporalmente cambios no críticos (nuevas features, refactors arriesgados) y priorizar investigar la causa del consumo acelerado — no es una decisión de gusto personal, es consecuencia directa y objetiva del dato.

Taller en grupo — Definir SLO con IA

Modalidad: parejas, 20 minutos, Nivel de IA: 2 (borrador).

Prompt sugerido: "Para el endpoint POST /pedidos de nuestra Saga, propone un SLI preciso, un SLO razonable con su ventana de tiempo, y calcula qué error budget implica en número absoluto de solicitudes fallidas permitidas, asumiendo 10.000 pedidos al mes."

Verificación obligatoria: verifiquen a mano el cálculo aritmético del error budget en número absoluto — no acepten el número de la IA sin recalcularlo.

10.5. Correlación de trazas y debugging distribuido

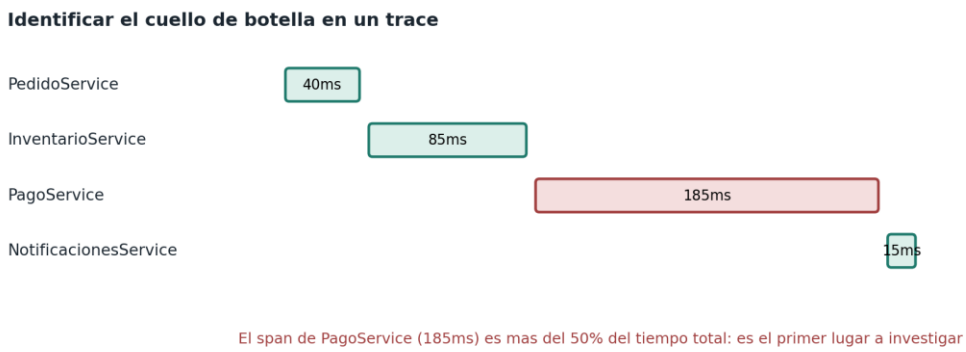
El trace_id como hilo conductor

Todo lo desarrollado converge en una habilidad práctica: usar el trace_id para reconstruir sin ambigüedad el recorrido de una solicitud. La disciplina consiste en dos prácticas complementarias: (1) incluir el trace_id en cada línea de log de cada servicio, creando un puente bidireccional entre Jaeger y los logs; (2) propagar ese mismo trace_id de servicio a servicio, para que todos los tramos de una solicitud compartan un único identificador — sin esto, cada servicio produce trazas huérfanas imposibles de correlacionar.

Identificar el cuello de botella

La vista principal de Jaeger para un trace individual es, en esencia, un diagrama de Gantt: el eje horizontal es tiempo, cada fila es un span, y la posición/ancho de la barra representan cuándo empezó, cuándo terminó y cuánto duró. Identificar un cuello de botella es, la mayoría de las veces, buscar la barra más larga — ese span es, por definición, el que más contribuyó a la duración total. Si ese span tiene hijos, hay que seguir bajando en el árbol hasta el span hoja que realmente consumió el tiempo.

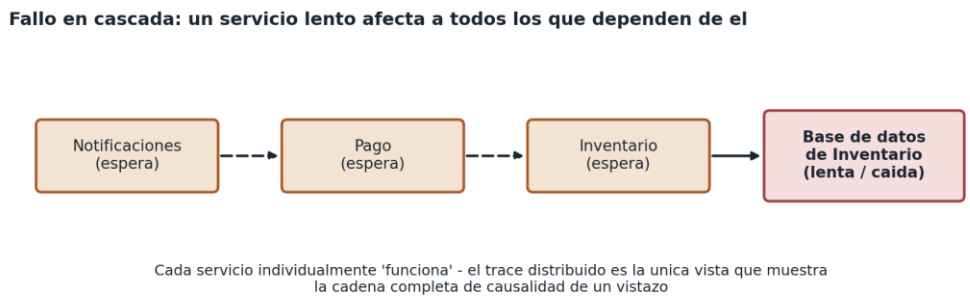
Diagrama 6. Identificar el cuello de botella dentro de un trace: el span más largo (rojo) es el primer lugar a investigar.



Diagnosticar un fallo en cascada

Un patrón valioso de detectar es el fallo en cascada: un servicio downstream falla o se vuelve lento, y ese retraso/error se propaga hacia arriba, afectando a todos los servicios que dependían de él, aunque cada uno individualmente "funcione bien". En una traza, se ve como varios spans consecutivos que terminan casi al mismo tiempo, todos con error o duración anormal, todos esperando al mismo span raíz del problema. Sin una traza unificada, cada equipo vería "mi servicio está lento" sin saber si la causa está en su propio código o río abajo.

Diagrama 7. Fallo en cascada: un servicio lento afecta a todos los que dependen de él; el trace distribuido muestra la cadena completa de causalidad.



Aplicación concreta a la Saga de Pedidos

Con los cuatro microservicios instrumentados, un pedido exitoso produce un único trace con cuatro spans en secuencia:

```
Trace: 4bf92f3577b34da6a3ce929d0e0e4736  (duracion total: 340ms)
|- span: POST /pedidos               [PedidoService]        0ms   -> 40ms
|  \- span: publish PedidoCreado     [PedidoService]        38ms  -> 40ms
|- span: consume PedidoCreado        [InventarioService]    45ms  -> 130ms
|  \- span: publish InventarioRes.   [InventarioService]    125ms -> 130ms
|- span: consume InventarioReservado [PagoService]          135ms -> 320ms   <- el mas
largo
|  \- span: publish PagoProcesado    [PagoService]          315ms -> 320ms
\- span: consume PagoProcesado       [NotificacionesService]325ms -> 340ms
```

En este ejemplo, PagoService es responsable de 185ms de los 340ms totales — más de la mitad del tiempo total del flujo — y sería el primer lugar donde investigar una optimización, exactamente el hallazgo que reproduce el reto de la práctica de esta semana.

Caso de estudio: reconstrucción de un incidente paso a paso

Lunes, 10:14 a.m. Una alerta de Grafana se dispara: el burn rate del SLO de POST /pedidos está en 8x sobre la ventana de 1 hora. El ingeniero de guardia confirma: la latencia p95 subió de 400ms a 2.3 segundos hace 20 minutos, de forma sostenida.

10:17 a.m. Abre Jaeger, filtra por nestjs-pedido-service y POST /pedidos, ordena por duración descendente. Las 20 trazas más lentas comparten un patrón: todas superan 2 segundos, todas completaron la Saga sin error, y en las 20, el span más largo es consume InventarioReservado en PagoService.

10:22 a.m. Abre una traza en detalle. El span de PagoService dura 2.05 de los 2.3 segundos totales. Dentro de ese span, la auto-instrumentación generó un span hijo para la llamada HTTP saliente hacia la pasarela de pago externa simulada — ese span hijo dura 1.98 segundos por sí solo.

10:25 a.m. La pregunta cambia de "¿qué servicio es lento?" a "¿por qué la pasarela externa responde lento?" — ya no se responde con las trazas propias, sino investigando el estado del proveedor externo. El ingeniero documenta el hallazgo con el trace_id exacto y escala el problema, evitando revisar sin rumbo el código de los cuatro microservicios propios.

Sin trazas distribuidas, la investigación hubiera empezado revisando logs desordenados de cuatro servicios, sin pista clara de por dónde empezar, y muy probablemente hubiera tardado horas en vez de los 11 minutos que tomó aquí desde la alerta hasta la causa raíz exacta.

Ejercicio resuelto

Enunciado: abres un trace en Jaeger y ves que el span raíz dura 2.1 segundos, pero la suma de las duraciones de sus spans hoja suma solo 400ms. ¿Qué hipótesis investigar primero?

Solución: hay 1.7 segundos no explicados por trabajo activo instrumentado. Hipótesis: (1) tiempo de espera en una cola (no aparece como span de trabajo, sino como el hueco entre el fin de un span y el inicio del siguiente), (2) una operación sin instrumentar, (3) contención de recursos (CPU/memoria). La traza sola no distingue entre estas tres — hay que cruzarla con métricas de infraestructura y tamaño de cola.

Ejercicio integrador de cierre

Enunciado: Retoma el ejemplo de la traza de 340ms de la Saga de Pedidos usado a lo largo de esta sección. Supón ahora que, tras desplegar un cambio en PagoService, la latencia de ese span sube de 185ms a 900ms de forma consistente en TODAS las trazas nuevas (no solo algunas), mientras que el resto de los spans (PedidoService, InventarioService, NotificacionesService) se mantienen exactamente igual que antes. Responde, en orden: (a) ¿qué pilar de observabilidad usarías primero para DETECTAR este problema, antes de siquiera abrir Jaeger? (b) una vez detectado, ¿qué harías en Jaeger para confirmar que el aumento está localizado específicamente en PagoService y no distribuido entre los cuatro servicios? (c) ¿qué acción de negocio (relacionada con SLO/error budget) tomarías si este cambio se mantiene así durante 48 horas? (d) ¿qué diferencia esta situación de un fallo en cascada?

Solución: (a) Una métrica: un dashboard de Grafana con la latencia p95 por servicio mostraría el salto de PagoService de forma inmediata y barata, sin necesidad de inspeccionar trazas individuales una por una — las métricas son la primera línea de detección, no de diagnóstico detallado. (b) En Jaeger, filtrarías por el servicio PagoService y la operación específica, y compararías la distribución de duraciones antes/después del despliegue (Jaeger permite comparar por rango de tiempo); confirmarías que el span de PagoService concentra el 100% del incremento observando que los demás spans del mismo trace mantienen su duración habitual. (c) Si el SLO de la Saga completa es, por ejemplo, "p95 < 2 segundos", y el nuevo total (340ms - 185ms + 900ms ≈ 1055ms) todavía está dentro del SLO, técnicamente no se ha incumplido el objetivo — pero un ingeniero prudente igual investigaría la causa del aumento de 5x en un solo servicio, porque reduce el margen del error budget disponible para absorber cualquier otro problema simultáneo; si el SLO fuera más estricto (por ejemplo, p95 < 800ms) ya estaría

formalmente incumplido y ameritaría revertir el despliegue de inmediato. (d) Esto NO es un fallo en cascada: en un fallo en cascada, el problema se origina en un servicio y se propaga causando que otros servicios TAMBIÉN se vuelvan lentos o fallen (como en el Diagrama 7). Aquí, el problema está contenido dentro de un único servicio, sin afectar el comportamiento de los demás — es una regresión de rendimiento localizada, un escenario más simple de diagnosticar precisamente porque la traza muestra con claridad que no hay propagación del problema hacia otros spans.

Taller en grupo — Diagnosticar con IA a partir de una descripción de spans
Modalidad: parejas, 25 minutos, Nivel de IA: 3 (revisor).

Prompt sugerido: "Dado este árbol de spans con sus duraciones [pega la descripción], identifica el cuello de botella real, propone dos hipótesis de causa, y para cada una indica qué dato adicional confirmaría o descartaría esa hipótesis."

Verificación obligatoria: para cada hipótesis, confirmen si el dato adicional sugerido realmente existe o sería fácil de obtener en su propio sistema.

Autoevaluación: preguntas de repaso

Antes de pasar a la práctica, responde estas preguntas sin ver el material — luego verifica tu respuesta contra la explicación breve que la acompaña. Este ejercicio de autoevaluación es una técnica de estudio deliberadamente más efectiva que releer pasivamente el capítulo (recuperación activa, o "retrieval practice", una de las técnicas de estudio con mayor evidencia empírica de efectividad).

1. ¿Por qué un sistema puede tener un monitoreo excelente y aun así no ser observable?

Porque el monitoreo solo responde preguntas anticipadas de antemano mediante métricas predefinidas; la observabilidad exige poder responder preguntas que nadie anticipó, lo cual requiere datos crudos ricos (trazas y logs estructurados), no solo dashboards de métricas fijas.

2. ¿Qué información pierdes si solo tienes métricas agregadas y ningún log ni traza?

Pierdes la capacidad de identificar CUÁL solicitud individual causó una anomalía y POR QUÉ — las métricas agregadas muestran tendencias pero no el detalle de un caso particular.

3. ¿Qué evita que dos servicios que se comunican por una cola de mensajes (como RabbitMQ) terminen en el mismo trace?

Que el productor no propague el contexto de traza (el traceparent) en los headers del mensaje. Sin esa propagación manual o automática, el consumidor genera un trace_id nuevo, desconectado del original.

4. ¿Por qué el p99 de latencia es más relevante que el promedio para la experiencia real de usuario?

Porque el promedio oculta los valores extremos; un p99 alto significa que el 1% de las solicitudes (una cantidad enorme en volumen absoluto) tiene una experiencia mucho peor que la que sugiere el promedio.

5. Si tu error budget mensual ya se agotó al día 10, ¿qué decisión de ingeniería es consistente con la práctica SRE?

Congelar temporalmente cambios no críticos y priorizar estabilizar el sistema — no es una decisión subjetiva, es la consecuencia directa de haber definido el error budget de antemano.

6. ¿Qué diferencia a head-based sampling de tail-based sampling?

Head-based decide si exportar una traza al inicio, sin saber cómo terminará; tail-based decide al final, permitiendo reglas como 'conserva siempre las trazas con error', más costoso pero de mayor valor diagnóstico.

7. En un trace con varios spans anidados, ¿cómo identificas el cuello de botella real?

Buscando la barra (span) más larga en la vista tipo Gantt; si ese span tiene hijos, hay que seguir bajando en el árbol hasta encontrar el span hoja que realmente consumió el tiempo.

8. ¿Por qué OpenTelemetry, siendo un estándar, facilita cambiar de proveedor de observabilidad más adelante?

Porque la instrumentación (SDK + API) es la misma independientemente del backend; cambiar de proveedor es, idealmente, solo cambiar la configuración del exporter, sin reinstrumentar el código de la aplicación.

9. ¿Qué patrón revela una traza distribuida que sería casi invisible revisando logs de un solo servicio a la vez?

Un fallo en cascada: varios servicios lentos o con error simultáneamente, todos esperando a un mismo servicio downstream — solo visible como una cadena de causalidad completa dentro de una única traza.

10. ¿Por qué incluir el trace_id en cada línea de log, y no solo en los spans?

Porque crea un puente bidireccional: desde una traza en Jaeger puedes buscar ese trace_id en los logs para obtener el detalle textual completo (un mensaje de error, una pila de llamadas) que un span, por su naturaleza estructurada, no siempre captura con la misma riqueza.

Tabla resumen de la semana

Concepto	En una frase
Observabilidad	Poder responder preguntas no anticipadas sobre el comportamiento de un sistema.
OpenTelemetry	Estándar abierto vendor-neutral para generar y exportar logs, métricas y trazas.
Trace / Span	El recorrido completo de una solicitud, dividido en unidades de trabajo con duración exacta.
Jaeger	Backend de visualización de trazas distribuidas (recepción, consulta, UI tipo Gantt).
SLI / SLO / SLA	Indicador medible -> objetivo interno -> compromiso contractual externo.
Error budget	Margen de fallo permitido antes de incumplir un SLO.
Sampling	Estrategia para no trazar el 100% de las solicitudes en sistemas de alto volumen.
Debugging distribuido	Usar el trace_id y la duración de cada span para localizar cuellos de botella y fallos en cascada.

Conclusión de la semana

La observabilidad no es una casilla que se marca instalando una herramienta — es una disciplina que se construye deliberadamente, decisión a decisión: qué se instrumenta y qué no, qué se propaga y qué se pierde en cada salto asíncrono, qué SLO refleja de verdad lo que le importa al negocio y cuál es solo un número arbitrario. Los cinco temas de esta semana no son piezas independientes de trivia técnica: forman un único argumento. Los tres pilares (10.1) son la materia prima; OpenTelemetry (10.2) es el mecanismo estandarizado para producirla sin atarte a un proveedor; Jaeger, ELK y Grafana (10.3) son donde esa materia prima se vuelve visible para un humano; los SLI/SLO/error budget (10.4) son el puente que conecta esa visibilidad técnica con decisiones de negocio defendibles; y la correlación de trazas (10.5) es la habilidad concreta, del día a día, que convierte todo lo anterior en la capacidad real de resolver un incidente en minutos en vez de horas.

El proyecto Amazon Distribuido, la Saga de Pedidos de la Semana 6, y en general cualquier sistema que construyas de aquí en adelante en tu carrera profesional, tarde o temprano va a fallar de una forma que nadie anticipó al escribir el código original. La diferencia entre un equipo que resuelve ese fallo en 15 minutos y uno que tarda 4 horas casi nunca está en la inteligencia individual de sus ingenieros — está, de forma sistemática y medible, en si el sistema fue construido desde el principio para ser observado, o si la observabilidad se intentó añadir apresuradamente después de que el incidente ya estaba en curso.

Preguntas frecuentes

¿Necesito instalar Prometheus y Grafana para completar la práctica de esta semana? No. La práctica se centra en trazas (Jaeger) por ser el hilo conductor de la semana; Prometheus/Grafana se describen conceptualmente en 10.3 como parte del stack completo, pero no son un requisito técnico del laboratorio.

¿Qué pasa si mi trace distribuido muestra solo 3 de los 4 microservicios? Casi siempre significa que el cuarto servicio no propagó o no extrajo correctamente el contexto de traza en los headers del mensaje de RabbitMQ — revisa el Paso 5 de la práctica antes de asumir que es un problema de Jaeger.

¿Es necesario entender matemáticamente el cálculo de percentiles para aprobar esta semana? No a nivel de fórmula estadística exacta, pero sí es indispensable entender POR QUÉ un percentil alto (p95, p99) es más informativo que un promedio para decisiones de negocio sobre experiencia de usuario — es el concepto, no el cálculo exacto, lo que se evalúa.

Anexo: comandos y configuración de referencia rápida

Este anexo reúne, en un solo lugar, los comandos y fragmentos de configuración que usarás en la práctica de esta semana — útil como referencia rápida durante el laboratorio, sin tener que releer toda la teoría.

Levantar Jaeger all-in-one con Docker

```
docker run -d --name jaeger \
  -p 16686:16686 \
  -p 4318:4318 \
  jaegertracing/all-in-one:latest

# UI:            http://localhost:16686
# OTLP HTTP in:  http://localhost:4318/v1/traces
```

Dependencias del SDK de OpenTelemetry para Node.js/NestJS

```
npm install @opentelemetry/sdk-node \
  @opentelemetry/auto-instrumentations-node \
  @opentelemetry/exporter-trace-otlp-http \
  @opentelemetry/resources \
  @opentelemetry/semantic-conventions
```

Variables de entorno útiles de OpenTelemetry

```
OTEL_SERVICE_NAME=nestjs-pedido-service
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
OTEL_TRACES_SAMPLER=always_on   # 100% de muestreo, solo para desarrollo/aula
```

Errores comunes al instrumentar por primera vez

La siguiente lista recoge los errores más frecuentes que cometen los equipos la primera vez que instrumentan un sistema con OpenTelemetry, observados de forma consistente en cohortes anteriores de este curso — revisarla antes de empezar la práctica ahorra, en promedio, la mitad del tiempo de depuración del laboratorio:

- Importar tracing.ts DESPUÉS de otros módulos en main.ts: la auto-instrumentación no captura nada porque los módulos ya se cargaron sin el parche activo.
- Olvidar levantar Jaeger antes de arrancar los microservicios: el exporter falla silenciosamente al no encontrar el endpoint OTLP, y ningún trace llega a la UI sin que aparezca un error evidente en la consola de la aplicación.
- Usar el mismo service.name en los 4 microservicios: Jaeger los trata como una sola entidad, haciendo imposible distinguir a cuál pertenece cada span en la vista de servicios.
- Asumir que la propagación de contexto en RabbitMQ es automática sin verificarlo: algunas versiones de librerías de mensajería no propagan el traceparent por defecto, y el síntoma (trazas separadas por servicio) es idéntico al de no haber instrumentado en absoluto.
- Cerrar la terminal de un microservicio antes de revisar Jaeger: sin logs visibles en consola, se pierde la mitad de la información necesaria para correlacionar un trace con su detalle textual.

Checklist de verificación de instrumentación

- El archivo tracing.ts se importa como la PRIMERA línea de main.ts, antes de cualquier otro import.

- El nombre de servicio (service.name) es único y descriptivo por cada uno de los 4 microservicios de la Saga.
- Al hacer una petición de prueba, aparece un trace nuevo en http://localhost:16686 en menos de unos segundos.
- Un trace que atraviesa RabbitMQ muestra spans de más de un servicio bajo el MISMO trace_id, no varios trace_id sueltos.
- Cada línea de log de cada microservicio incluye el trace_id o correlation_id vigente en ese momento.

Glosario de términos

- Observabilidad: capacidad de un sistema de exponer suficiente información interna para permitir responder preguntas no anticipadas sobre su comportamiento.
- Trace: el recorrido completo de una solicitud a través de un sistema, identificado por un trace_id único.
- Span: una unidad de trabajo dentro de un trace, con inicio, fin, atributos y, opcionalmente, un span padre.
- SLI (Service Level Indicator): un indicador medible derivado de datos de observabilidad.
- SLO (Service Level Objective): un objetivo interno definido sobre un SLI.
- SLA (Service Level Agreement): un compromiso contractual externo, generalmente más laxo que el SLO interno.
- Error budget: el margen de incumplimiento que un SLO permite antes de considerarse violado.
- W3C Trace Context: estándar que define cómo se codifica y propaga el contexto de una traza (header traceparent).
- Collector (OpenTelemetry Collector): proceso intermedio que recibe, procesa y reenvía datos de observabilidad.
- Fallo en cascada: patrón en el que el fallo o lentitud de un servicio downstream se propaga a los servicios que dependen de él.
- Burn rate: la velocidad, relativa al ritmo presupuestado, a la que se consume un error budget.
- Sampling (muestreo): estrategia para exportar solo una fracción de las trazas generadas, reduciendo costo de almacenamiento y transmisión.
- SpanKind: clasificación del rol de un span (SERVER, CLIENT, PRODUCER, CONSUMER) dentro de una interacción distribuida.
- Convenciones semánticas: nombres estandarizados de atributos comunes en OpenTelemetry (http.method, db.system, etc.) reconocidos automáticamente por herramientas de análisis.

Referencias

OpenTelemetry. (2024). OpenTelemetry Documentation. https://opentelemetry.io/docs/ — Documentación oficial del SDK, la API y las convenciones semánticas usadas en la instrumentación de la práctica de esta semana.

Jaeger. (2024). Jaeger Tracing Documentation. https://www.jaegertracing.io/docs/ — Referencia de la arquitectura de Jaeger, incluida la imagen all-in-one usada en el laboratorio.

Grafana Labs. (2024). Grafana y Prometheus Documentation. https://grafana.com/docs/ — Guía de referencia para construir dashboards y definir reglas de alerta basadas en burn rate.

Elastic. (2024). Elastic Stack (ELK) Documentation. https://www.elastic.co/guide/ — Documentación del stack de centralización y búsqueda de logs.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media. — La referencia fundacional de SLI/SLO/error budget y las alertas basadas en burn rate citadas en la sección 10.4.

Sridharan, C. (2018). Distributed Systems Observability. O'Reilly Media. — Desarrolla en profundidad la distinción entre monitoreo y observabilidad presentada en 10.1.

W3C. (2021). Trace Context Recommendation. https://www.w3.org/TR/trace-context/ — La especificación formal del header traceparent usado para la propagación de contexto entre servicios.